

EKS Best Practice: Public NLB for ingress-nginx — Without AWS Load Balancer Controller

Cluster: "test-cluster" · Region: us-east-2 · Ingress controller: NGINX, listening on 80/443

Recommended architecture

Internet → Network Load Balancer (manually provisioned, instance target type) → Worker Node Auto Scaling Group (fixed NodePorts) → kube-proxy → ingress-nginx pods → backend Services

Without the AWS Load Balancer Controller, nothing inside the cluster watches Kubernetes objects and calls the ELBv2 API for you — the NLB is provisioned and wired up directly with the AWS CLI (or your existing Terraform/IaC pipeline), and it targets **EC2 instances**, not pod IPs. That means the ingress-nginx Service must be exposed as **NodePort** rather than via pod-IP registration.

This also solves the earlier FailedScheduling ... didn't have free ports error if ingress-nginx is currently using `hostNetwork` / `hostPort`. A NodePort is opened by kube-proxy on every node automatically, and kube-proxy forwards incoming traffic to any healthy nginx pod cluster-wide — regardless of which node that pod actually landed on. There's no "one pod per node per port" restriction the way `hostPort` has, so nginx can scale normally.

Why go this route instead of the AWS Load Balancer Controller

- No extra in-cluster controller needing broad IAM permissions over ELBv2/EC2/ACM/WAF — smaller IAM and RBAC footprint.
- Load balancer lifecycle lives entirely in your existing infrastructure-as-code / CI pipeline, audited the same way as the rest of your AWS resources.
- One fewer moving part in the cluster to upgrade, patch, and monitor.
- Trade-off: target registration is at the *node* level (via the Auto Scaling Group), not the pod level, and any change to the LB (listeners, health checks, certs) is a CLI/IaC change rather than a Kubernetes annotation.

Step 1 · Expose ingress-nginx as a NodePort Service with fixed ports

```
apiVersion: v1
kind: Service
metadata:
  name: ingress-nginx-controller
  namespace: ingress-nginx
spec:
  type: NodePort
  ports:
    - name: http
      port: 80
      targetPort: http
      nodePort: 30080
    - name: https
      port: 443
      targetPort: https
      nodePort: 30443
  selector:
    app.kubernetes.io/name: ingress-nginx
    app.kubernetes.io/component: controller
```

```
kubectl apply -f ingress-nginx-nodeport-service.yaml
kubectl get svc -n ingress-nginx ingress-nginx-controller
```

Fixing the `nodePort` values (rather than letting Kubernetes pick from the ephemeral range) keeps them stable across redeloys, which matters because the target group below points at these exact ports.

Step 2 · Gather the IDs you'll need

```
CLUSTER_NAME=test-cluster
REGION=us-east-2

VPC_ID=$(aws eks describe-cluster --name $CLUSTER_NAME --region $REGION \
  --query "cluster.resourcesVpcConfig.vpcId" --output text)

# Public subnets to place the NLB in (adjust the filter/tag to match your VPC)
SUBNET_IDS=$(aws ec2 describe-subnets --region $REGION \
  --filters "Name=vpc-id,Values=$VPC_ID" "Name=tag:kubernetes.io/role/elb,Values=1" \
  --query "Subnets[].SubnetId" --output text)
echo $SUBNET_IDS

# Managed node group's underlying Auto Scaling Group name
NODEGROUP_NAME=$(aws eks list-nodegroups --cluster-name $CLUSTER_NAME --region $REGION \
  --query "nodegroups[0]" --output text)
```

```

ASG_NAME=$(aws eks describe-nodegroup --cluster-name $CLUSTER_NAME \
  --nodegroup-name $NODEGROUP_NAME --region $REGION \
  --query "nodegroup.resources.autoScalingGroups[0].name" --output text)
echo $ASG_NAME

# Node security group (adjust if your nodes use a group other than the cluster's shared SG)
NODE_SG_ID=$(aws eks describe-cluster --name $CLUSTER_NAME --region $REGION \
  --query "cluster.resourcesVpcConfig.clusterSecurityGroupId" --output text)
echo $NODE_SG_ID

```

Step 3 • Create target groups (instance target type)

```

HTTP_TG_ARN=$(aws elbv2 create-target-group \
  --name test-cluster-nginx-http \
  --protocol TCP --port 30080 \
  --vpc-id $VPC_ID --target-type instance \
  --health-check-protocol TCP --health-check-port 30080 \
  --region $REGION \
  --query "TargetGroups[0].TargetGroupArn" --output text)

HTTPS_TG_ARN=$(aws elbv2 create-target-group \
  --name test-cluster-nginx-https \
  --protocol TCP --port 30443 \
  --vpc-id $VPC_ID --target-type instance \
  --health-check-protocol TCP --health-check-port 30443 \
  --region $REGION \
  --query "TargetGroups[0].TargetGroupArn" --output text)

```

Step 4 • Create the Network Load Balancer

```

NLB_ARN=$(aws elbv2 create-load-balancer \
  --name test-cluster-public-nlb \
  --type network \
  --scheme internet-facing \
  --subnets $SUBNET_IDS \
  --region $REGION \
  --query "LoadBalancers[0].LoadBalancerArn" --output text)

NLB_DNS=$(aws elbv2 describe-load-balancers --load-balancer-arns $NLB_ARN --region $REGION \
  --query "LoadBalancers[0].DNSName" --output text)
echo $NLB_DNS

```

Step 5 • Create listeners

```

# Plain TCP passthrough on 80 and 443 (TLS terminated by nginx inside the cluster)
aws elbv2 create-listener \
  --load-balancer-arn $NLB_ARN \
  --protocol TCP --port 80 \
  --default-actions Type=forward,TargetGroupArn=$HTTP_TG_ARN \
  --region $REGION

aws elbv2 create-listener \
  --load-balancer-arn $NLB_ARN \
  --protocol TCP --port 443 \
  --default-actions Type=forward,TargetGroupArn=$HTTPS_TG_ARN \
  --region $REGION

```

To terminate TLS at the NLB instead (simpler cert rotation, but nginx then receives decrypted HTTP), create the 443 listener as `--protocol TLS` with `--certificates CertificateArn=<ACM_CERT_ARN>` pointed at the HTTP target group instead.

Step 6 • Attach the node group's Auto Scaling Group to both target groups

This is what replaces the AWS Load Balancer Controller's job: instead of a controller registering targets automatically, the ASG itself keeps the target groups in sync as nodes scale up, scale down, or get replaced.

```

aws autoscaling attach-load-balancer-target-groups \
  --auto-scaling-group-name $ASG_NAME \
  --target-group-arns $HTTP_TG_ARN $HTTPS_TG_ARN \
  --region $REGION

```

Step 7 • Allow the NodePort range on the node security group

Because there's no controller-managed security group on the NLB side, and NLBs are pass-through (the client's real source IP reaches the node), the node security group itself must allow inbound traffic on the NodePorts from the internet:

```

aws ec2 authorize-security-group-ingress \
  --group-id $NODE_SG_ID \
  --protocol tcp --port 30080 --cidr 0.0.0.0/0 \

```

```
--region $REGION
```

```
aws ec2 authorize-security-group-ingress \  
  --group-id $NODE_SG_ID \  
  --protocol tcp --port 30443 --cidr 0.0.0.0/0 \  
  --region $REGION
```

Step 8 • Enable cross-zone load balancing

```
aws elbv2 modify-load-balancer-attributes \  
  --load-balancer-arn $NLB_ARN \  
  --attributes Key=load_balancing.cross_zone.enabled,Value=true \  
  --region $REGION
```

Without this, the NLB only balances traffic within the AZ that received the connection, which can unevenly load nodes/pods if your AZs have different node counts.

Step 9 • Verify

```
aws elbv2 describe-target-health --target-group-arn $HTTP_TG_ARN --region $REGION  
aws elbv2 describe-target-health --target-group-arn $HTTPS_TG_ARN --region $REGION  
  
curl -I http://$NLB_DNS
```

Give it a couple of minutes after attaching the ASG — existing instances register automatically, but health checks need a few cycles to pass before targets show `healthy`.

Step 10 • Point DNS at the NLB

```
NLB_ZONE_ID=$(aws elbv2 describe-load-balancers --load-balancer-arns $NLB_ARN --region $REGION \  
  --query "LoadBalancers[0].CanonicalHostedZoneId" --output text)  
  
aws route53 change-resource-record-sets \  
  --hosted-zone-id \  
  --change-batch '{  
    "Changes": [{  
      "Action": "UPSERT",  
      "ResourceRecordSet": {  
        "Name": "app.example.com",  
        "Type": "A",  
        "AliasTarget": {  
          "HostedZoneId": "'${NLB_ZONE_ID}'",  
          "DNSName": "'${NLB_DNS}'",  
          "EvaluateTargetHealth": true  
        }  
      }  
    }]  
  }'
```

Best-practice checklist

- **NodePort, not hostNetwork/hostPort** — kube-proxy exposes the port on every node uniformly, so pods aren't pinned one-per-node and scheduling isn't constrained by port availability.
- **Fix the NodePort values explicitly** in the Service spec so they stay stable across redeloys and always match the target group configuration.
- **Attach the ASG to the target groups** rather than registering instance IDs by hand — this is what keeps targets correct as the node group scales or cycles instances, without needing a cluster-side controller.
- **Cross-zone load balancing enabled** so traffic isn't skewed toward whichever AZ happens to receive the connection.
- **TCP health checks on the NodePort itself** keep things simple; only add an HTTP path-based check if you need finer-grained health signal than "is kube-proxy forwarding on this port."
- **Node security group must allow the NodePort range from the internet** since there's no controller-managed security group sitting in front — scope this down to just the two ports in use, not a wide range.
- **Re-run the ASG attachment (or manage it in Terraform) if the node group is ever replaced** — a brand-new node group means a new ASG, which needs to be attached to the target groups again.